

Vodafone Tourist Pack

Backend Documentation

Spring Boot REST API: pack catalog & custom pricing, PayPal-verified activation, eSIM provisioning, PassKit wallet passes, and the Daily Drop game.

Prepared for internship demo

September 2026

1. Project Overview

The Vodafone Tourist Pack backend is a Spring Boot 4.1 (Java 21) REST API serving the Tourist Pack frontend: the pack catalog and custom-plan pricing, PayPal-verified pack activation, demo eSIM provisioning with a real inline QR code in the confirmation email, Apple/Google Wallet pass issuance via PassKit, and the Daily Drop reward game.

2. Tech Stack

- Spring Boot 4.1, Java 21
- Spring Data JPA + PostgreSQL, hosted on Supabase
- Spring Validation, Spring Mail (Gmail SMTP)
- Lombok
- com.google.zxing - real eSIM activation QR codes rendered inline in the confirmation email
- bucket4j - in-memory per-IP rate limiting
- Hibernate ddl-auto=update - schema changes apply automatically on startup. There is no Flyway/Liquibase migration history.

3. Package Structure

controller/	REST layer - thin, delegates to services
service/	Business logic (ActivationService, GameService, EmailService, PassKitService, CustomPlanPricingService, QrCodeService, EsimProvisioningService, ...)
repository/	Spring Data JPA interfaces
model/	JPA entities = database tables
dto/	Record types for request/response bodies
config/	CorsConfig, RateLimitFilter

4. Database Entities

```
Tourist --< UserSubscription >-- Pack
Tourist --< PaymentTransaction
Tourist --< CreditTransaction
Tourist --< DailyCreditClaim
Tourist --< Gameplay >-- Game
          Gameplay >-- Prize
Pack --< PackFeature
Partner --< PartnerOffer
```

Currency note: Pack.priceAll is stored in Albanian Lek (ALL). The frontend converts it to EUR before creating the PayPal order.

5. API Endpoints

GET /api/v1/packs	Active fixed packs. GET /api/v1/packs/{id} also returns a hidden custom-built pack (is_active=false) - no active-only filter on the by-id lookup.
GET /api/v1/packs/sponsors	Partner offers shown on the landing page.
POST /api/v1/packs/custom/quote	Stateless live price for a data/minutes/duration combo - no database write.
POST /api/v1/packs/custom/build	Persists that combo as a real, hidden Pack row and returns it, so it flows through the same checkout pipeline as a fixed pack unchanged.
POST /api/v1/activations	The paid-activation endpoint. Requires PayPal order/capture proof; upserts the Tourist by email, creates the UserSubscription, records the payment, enrolls in PassKit, sends the welcome email.
GET /api/v1/tourists/{id}/subscription	Everything /my-pack needs: pack details, key dates, order info, game credits, wallet links.
GET /api/v1/tourists/discount-by-email	Looks up an unredeemed game-hub discount, used by the frontend's PayPal create-order route to apply it to the price it quotes PayPal.
Game hub endpoints	Idempotent daily credit claim (per tourist, per day, in the Europe/Tirane timezone), scratch-card play, prize catalog.

6. Payment Trust Model

POST /activations is called by the frontend's own Next.js server, never directly by the browser - it represents a pack that has already been paid for. The backend records the PayPal order id and capture id as proof of payment and guards against duplicate submissions via `PaymentTransactionRepository.findByPaypalOrderId()`.

The backend does not itself re-verify the charged amount against PayPal's API - that trust is established one layer up, in the frontend's create-order route, which always re-fetches the authoritative price from this backend rather than trusting a client-supplied number before ever creating the PayPal order.

7. Custom Plan Pricing - Deterministic by Design

CustomPlanPricingService computes price as a pure function of the request: a base fee, plus a per-GB rate (or a flat Unlimited-data rate), plus a per-100-minutes rate, plus a per-day rate, rounded to a clean multiple. No external calls, no AI/LLM involvement - reproducible, auditable, and cheap enough to call on every slider drag in the UI.

This was a deliberate architectural choice over an AI-generated price: a price has to be the same number every time for the same inputs, has to be instant, and has to be explainable if ever disputed. The constants in CustomPlanPricingRates are calibrated against the real fixed-pack prices but remain placeholders pending business sign-off.

8. PassKit Wallet Integration

PassKitService enrolls each activated tourist as a PassKit member so their pack appears as an Apple/Google Wallet pass. A real bug was found and fixed during development: enrollment called PUT /members/member, but PassKit's API only allows PUT to update an already-existing member - it returns 404 for anyone new. Every enrollment had been silently failing (caught and logged) and falling back to a dead local link. Confirmed against the live API and fixed by switching to POST, which creates.

Dynamic pass fields (data allowance, minutes, validity, a computed expiry date, and back-of-card links to the game hub and a data-usage page) are sent as PassKit member metaData on every enrollment. Binding a pass template field to one of those metaData keys is a one-time step in PassKit's own visual Pass Designer - not something this API can configure. The pass's QR code is generated automatically by PassKit from the member id; the backend never sets or overrides it.

9. Email Delivery

EmailService sends a branded HTML confirmation email over Gmail SMTP, with an inline header image and, for eSIM deliveries, a real QR code (rendered by QrCodeService using zxing) encoding the activation code generated by EsimProvisioningService - a demo-only service producing well-formed LPA activation data that iOS recognizes and opens its native eSIM-install screen for, without a real telecom SM-DP+ server behind it.

A failed email send (e.g. an SMTP outage) is logged but never rolls back an activation the tourist has already paid for.

10. Rate Limiting

RateLimitFilter applies an in-memory, per-client-IP Bucket4j token bucket to sensitive endpoints under /api/v1/** - tightest on the paid-activation endpoint, looser on read-only endpoints like the pack catalog. Because this runs as a single backend instance with no shared cache, these buckets are process-local; running more than one instance would need to move this to a shared store such as Redis.

11. Environment Variables

DB_URL / DB_USERNAME / Supabase PostgreSQL connection.

DB_PASSWORD

APP_CORS_ALLOWED_ORI Comma-separated list of allowed frontend origins.

GIN

APP_GAME_HUB_TIMEZON Timezone the Daily Drop's once-per-day claim resets in (Europe/Tirane).

E

APP_FRONTEND_BASE_UR Used to build links back to the frontend (wallet fallback, game hub, email).

L

PASSKIT_API_BASE_URL / Apple/Google Wallet pass issuance.

PASSKIT_API_TOKEN /

PASSKIT_PROGRAM_ID /

PASSKIT_TIER_ID /

PASSKIT_PASS_URL_HOST

MAIL_HOST / MAIL_PORT / Gmail SMTP for the confirmation email.

MAIL_USERNAME /

MAIL_PASSWORD

12. Known Gotchas

- ddl-auto=update never backfills existing rows. A new nullable column on a table that already has data comes back NULL for every existing row - a primitive Java field (boolean, int) cannot hold that and Hibernate throws on the next SELECT. This has bitten this repo more than once; use wrapper types (Boolean, Integer) for anything added to an existing table.
- PassKit's free/draft tier caps total passes issued and expires each one after 48 hours.
- No Flyway/Liquibase - there is no schema migration history, only whatever Hibernate infers from the current entity definitions at startup.
- Comparing a JPA enum column against a Java enum literal in JPQL can generate a Postgres type cast that does not match the column's actual native enum type name - resolved for the discount-redemption query by switching to a native SQL query with an explicit ::text cast on both sides.